Министерство образования и науки Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого
—
Институт компьютерных наук и кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 2

«РЕФАКТОРИНГ ПРОГРАММЫ»

по дисциплине «Объектно-ориентированное программирование»

Выполнил Волошкевич М.А.
студент гр.5151001/40202 <подпись>

Преподователь / Ерастов В.О.
ассистент <подпись>

Санкт-Петербург
2026г.

1. Цель работы.

Освоение особенностей этапов анализа предметной области и проектирования архитектуры программного обеспечения в объектной модели.

2. Задачи

В рамках выполнения лабораторной работы были решены следующие задачи:

- анализ предметной области программы «Судоку», реализованной в процедурной парадигме, с выделением ключевых абстракций;
- анализ недостатков архитектуры и кода исходного проекта;
- проектирование новой архитектуры программы в объектной парадигме (рефакторинг) с использованием UML-диаграммы классов, построенной в PlantUML;
- программная реализация новой архитектуры на языке Python (все методы снабжены аннотациями типов);
- реализация unit-тестов для основного функционала программы.

Выполнены дополнительные задания: 5.1 (дополнительные UML-диаграммы структуры: три диаграммы объектов) и 5.2 (дополнительные UML-диаграммы поведения: диаграмма активностей). Дополнительное задание 5.3 не выполнялось.

3. Ход работы

3.1. Описание исходного проекта

Исходный проект представляет собой игру «Судоку» на языке Python с графическим интерфейсом на библиотеке pygame. Он состоит из трёх файлов общим объёмом около 1540 строк: `sudoku.py` (1177 строк) содержит интерфейс и главный цикл, `solver.py` (97 строк) содержит проверку правил и решатель, `boards.py` (265 строк) содержит набор готовых головоломок.

Предметная область: игровое поле размером 9×9 разбито на девять блоков 3×3. Игрок заполняет пустые клетки цифрами от 1 до 9 так, чтобы цифры не повторялись в строке, столбце и блоке.

Функционал программы:

- заставка с анимацией сетки и переход в меню по клавише Enter;
- главное меню: «Play», «Solver», «Score board», «Exit»;
- выбор одного из четырёх уровней сложности (easy, medium, hard, very hard) и выдача случайной заготовленной головоломки;
- игровой экран: выбор клетки мышью, ввод цифр от 1 до 9 и очистка клавишей 0, подсветка ошибочных цифр красным цветом, таймер, зелёная сетка при корректном решении, звуки и фоновая музыка;
- автоматическое решение по пробелу: мгновенное («Solution») или с анимацией («Animation»);
- режим «Solver»: игрок вводит собственную головоломку, а программа её решает;
- всплывающие окна подтверждения выхода и выбора способа решения;
- экран «Score board» (в исходной программе это заглушка).

Важные особенности реализации: решение ищется перебором с возвратом (backtracking) без рекурсии; состояние программы хранится в глобальных булевых флагах (`inMainMenu`, `inPlay`, `inGamePlay`, `inSolver`, `inPopUp` и других); все надписи создаются заранее в виде отдельных глобальных переменных.

3.2. Недостатки исходного проекта

3.2.1. Недостатки архитектуры

- Отсутствуют абстракции предметной области: поле, партия, решатель, таймер и экраны не выделены в самостоятельные сущности, а поле представлено «голым» списком списков.
- Состояние программы описывается более чем пятнадцатью глобальными переменными (`inMainMenu`, `inPlay`, `inSolver`, `inPopUp`, `gameGridSwitch`, `resetTime`, `resetSolverBoards` и другими). Допустимые комбинации флагов нигде не зафиксированы, а переходы между экранами размазаны по главному циклу.
- Логика игры и отрисовка перемешаны: функции отрисовки (`gamePlay`, `solver`) изменяют состояние (`updateBoard`), функция `gridView` определяет победу, а `clockTime` зависит от флага `greyGridLines`, который выставляется при рисовании сетки.
- Три глобальных массива `sudokuBoard`, `userBoard` и `currentBoard` необходимо синхронно обновлять вручную; режим «Solver» подменяет их через `global`.
- Режимы «Партия» и «Решатель» реализованы копированием кода (функции `gamePlay` и `solver` почти идентичны), различия спрятаны в проверках `inGamePlay` и `inSolver`.
- Анимация автоматического решения (`animationAutoSolveBoard`) выполняется внутри обработчика события и блокирует главный цикл: окно не реагирует на события до окончания решения.

3.2.2. Недостатки качества кода

- Дублирование: алгоритм backtracking скопирован в `autosolve` и `animationAutoSolveBoard`; функция `display_brd` есть и в `boards.py`, и в `solver.py`; четыре одинаковых блока обработки выбора сложности; 36 глобальных переменных для надписей цифр (`oneText`, `oneBigText`, `oneRedText` и так далее) и цепочки из девяти `elif` для вывода цифр.

- Магические числа: координаты областей клика заданы константами в функции `cursor()` (например, `180 < x < 400 and 300 < y < 360`) отдельно от координат отрисовки, поэтому при смене разметки они расходятся.
- Функция `cursor()` на каждом кадре выполняет `copy.deepcopy` списка клеток.
- Ошибка в `displayUserNums`: координаты клетки определяются через `rows.index(item)` и `userBoard.index(rows)`, что возвращает первую клетку с таким значением, поэтому проверка конфликта выполняется не для той клетки.
- Непоследовательные соглашения: `possible(board, x, y, num)` принимает пару (x, y), а индексирует `board[y][x]`; используется импорт `from solver import *`; смешаны стили camelCase и snake_case.
- При выходе из приложения вызывается `pg.quit()` без выхода из цикла, после чего цикл продолжает работать с закрытой библиотекой; в `boards.py` используется «голый» `except`.
- Мелкие дефекты: таймер выводится без ведущего нуля («1:5»), переменная `madeMove` и звук `gameOverSound` не используются, в `solver.py` оставлен собственный пример запуска.
- Ошибки в данных: головоломки ESY 02 и HRD 03 содержат повторяющиеся подсказки (цифра 4 дважды в последнем столбце ESY 02, цифра 1 дважды в седьмом столбце HRD 03), поэтому не имеют решения.
- Отсутствуют аннотации типов, документация и автоматические тесты.

3.3. Архитектура программы в объектной парадигме

При проектировании выделено два слоя. Ядро (`Board`, `SudokuSolver`, `Game`, `GameTimer`, `BoardRepository`, `Difficulty`) не зависит от pygame, поэтому тестируется без графического окна. Слой интерфейса (`Assets`, `Button`, `Popup`, `GridRenderer`, `Screen`, `App`) использует ядро и отвечает только за отображение и ввод.

Применены следующие приёмы: наследование и полиморфизм (`Game` → `PuzzleGame` и `FreeSolverGame`; `Button` → `ScalingButton`

и `InvertingButton`; `Popup` → `ConfirmPopup` и `SolvePopup`; иерархия `Screen`), паттерн «Шаблонный метод» (`Game._solution_source`, `GameScreen._request_exit` и `_on_completed`), замена набора булевых флагов экранами с явными переходами, внедрение зависимостей (источник времени в `GameTimer`, генератор случайных чисел в `BoardRepository`).

Архитектура представлена двумя диаграммами классов: ядро (Рис. 1) и интерфейс (Рис. 2). Исходные тексты диаграмм на написаны с импользованием инструмента PlantUML.

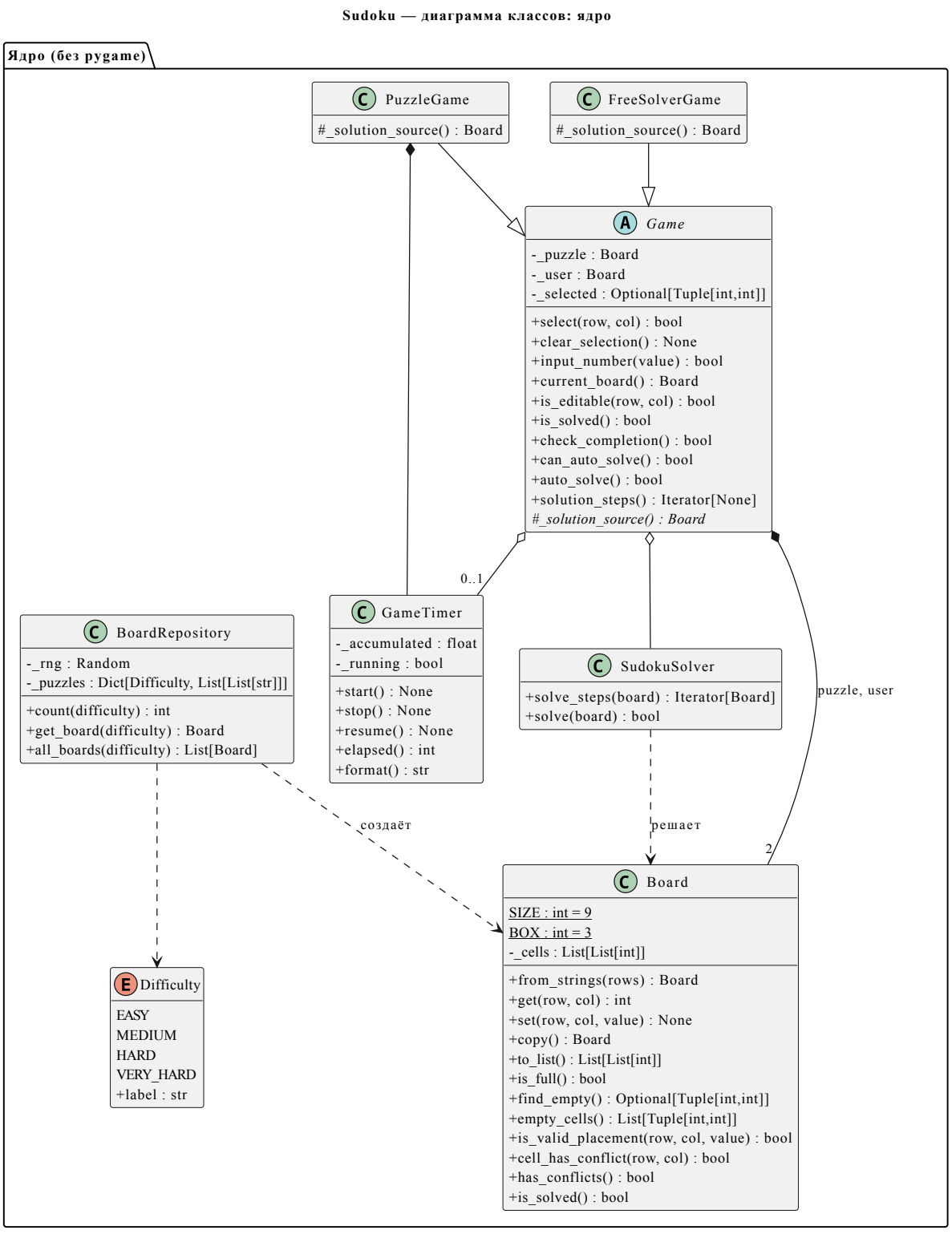


Рис. 1. UML-диаграмма классов: ядро (`class_diagram_core.puml`)

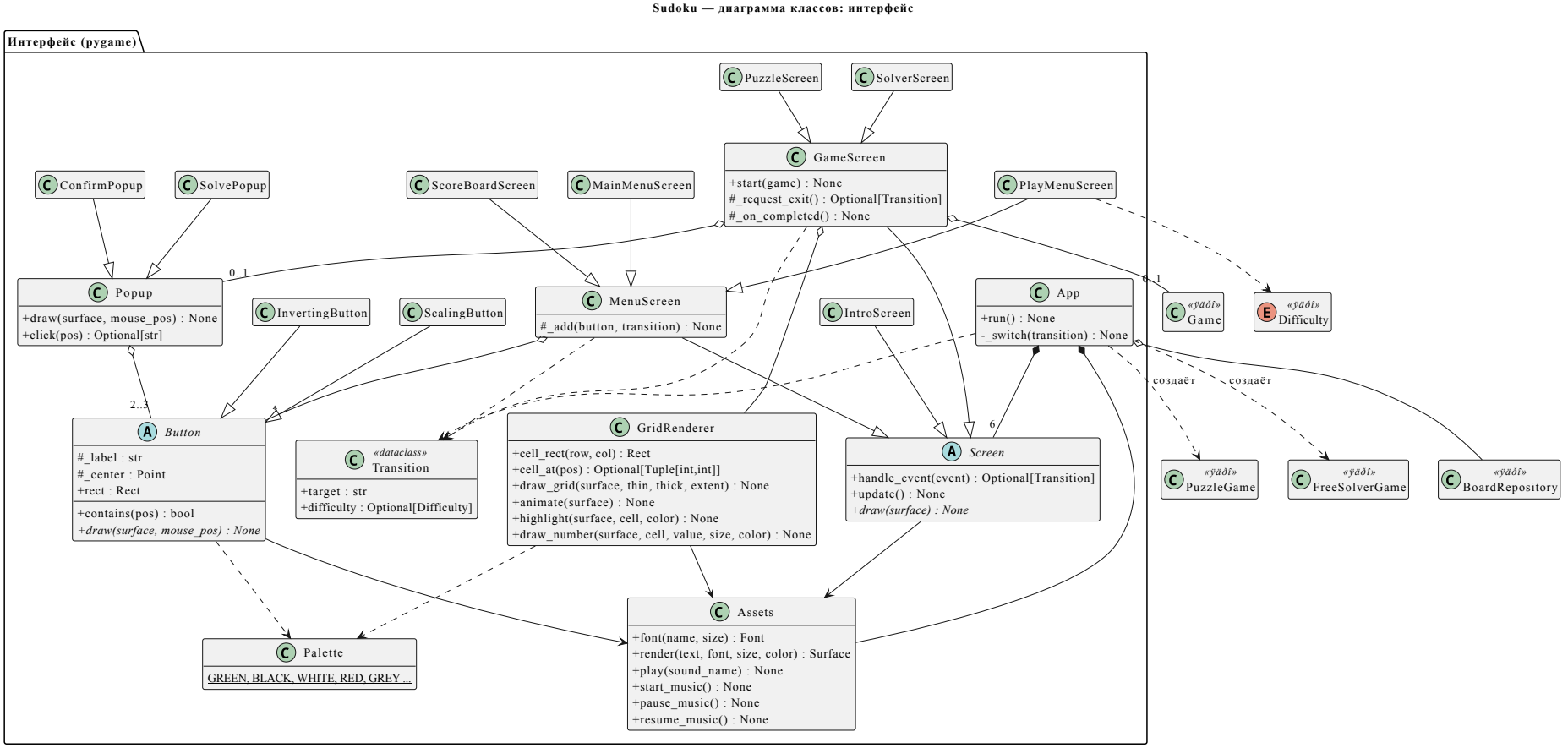


Рис. 2. UML-диаграмма классов: интерфейс (`class_diagram_ui.puml`)

Перечень классов и их назначение приведён в Таблица 1. Всего в архитектуре 28 классов.

Класс	**Файл**	**Назначение**
`Difficulty`	`difficulty.py`	Перечисление уровней сложности; подпись для кнопки меню.
`Board`	`board.py`	Поле 9×9: доступ к клеткам, копирование, поиск пустых клеток, проверка правил (строка, столбец, блок), обнаружение конфликтов, признак решённости.
`SudokuSolver`	`solver.py`	Решатель методом backtracking: мгновенное решение (solve) и пошаговое решение в виде генератора (solve_steps) для анимации.
`BoardRepository`	`board_repository.py`	Хранилище головоломок по уровням сложности; выдаёт копию случайной головоломки, отфильтровывая некорректные.
`GameTimer`	`timer.py`	Секундомер с остановкой и продолжением; источник времени внедряется снаружи (удобно для тестов).
`Game (абстрактный)`	`game.py`	Игровая сессия: исходное поле, ввод игрока, выбор клетки, проверка завершения, автоматическое решение. Источник решения определяет подкласс.
`PuzzleGame`	`game.py`	Обычная партия: решение строится из исходных подсказок, есть таймер.
`FreeSolverGame`	`game.py`	Режим «Solver»: решение строится из того, что ввёл игрок; таймера нет.
`Palette`	`ui.py`	Цветовые константы приложения.
`Assets`	`ui.py`	Загрузка и кеширование шрифтов, надписей, звуков и музыки.
`Button (абстрактный)`	`ui.py`	Кнопка-надпись: область клика совпадает с областью отрисовки.
`ScalingButton`	`ui.py`	Кнопка меню, увеличивающаяся при наведении.
`InvertingButton`	`ui.py`	Малая кнопка с инверсией цветов при наведении.
`Popup`	`ui.py`	Модальное окно с вопросом и набором кнопок; возвращает имя нажатого действия.
`ConfirmPopup`	`ui.py`	Подтверждение выхода в меню (cancel, ok).
`SolvePopup`	`ui.py`	Выбор способа решения (cancel, solution, animation).
`GridRenderer`	`ui.py`	Рисует сетку, подсветку и числа; переводит координаты мыши в клетку.
`Transition`	`screens.py`	Запрос перехода на другой экран (целевой экран, сложность).

`Screen (абстрактный)`	`screens.py`	Общий интерфейс экрана: handle_event, update, draw.
`IntroScreen`	`screens.py`	Заставка с анимацией сетки.
`MenuScreen`	`screens.py`	Экран-меню: заголовок и список кнопок с переходами.
`MainMenuScreen`	`screens.py`	Главное меню.
`PlayMenuScreen`	`screens.py`	Выбор сложности.
`ScoreBoardScreen`	`screens.py`	Экран таблицы рекордов (заглушка, как в исходной программе).
`GameScreen`	`screens.py`	Экран игрового поля: ввод, отрисовка, всплывающие окна, анимация решения по кадрам (не блокирует цикл).
`PuzzleScreen`	`screens.py`	Партия: подтверждение выхода и звук победы.
`SolverScreen`	`screens.py`	Режим решателя: выход в меню без подтверждения.
`App`	`app.py`	Создаёт окно и экраны, содержит главный цикл, переключает экраны, управляет музыкой.

Таблица 1. Классы программы

3.4. Особенности программной реализации

- Все методы и функции снабжены аннотациями типов (type hinting), в том числе `__init__` (`-> None`) и методы-генераторы (`Iterator[...]`).
- Булевы флаги состояния заменены экранами: текущий экран хранит `App`, а экран возвращает объект `Transition`, не зная о существовании других экранов.
- Backtracking реализован один раз в `SudokuSolver.solve_steps` как генератор. Мгновенное решение (`solve`) исчерпывает генератор, а анимация вызывает его по шагам: `GameScreen` делает по 20 шагов за кадр и не блокирует цикл событий.
- Различия между партией и режимом решателя вынесены в подклассы (шаблонный метод `Game._solution_source`): `PuzzleGame` решает от исходных подсказок и поэтому не зависит от ошибок игрока, `FreeSolverGame` решает от цифр, введённых игроком.
- Область клика кнопки вычисляется из тех же размеров надписи, что используются при отрисовке; координаты клеток вычисляет единственный класс `GridRenderer`.

- Вместо 36 переменных для цифр используется кеш надписей `Assets.render(text, font, size, color)`.
- Победа определяется одним методом `Board.is_solved` (поле заполнено и нет конфликтов); от него зависят цвет сетки, остановка таймера и звук. Таймер останавливается при победе и продолжается, если игрок нарушил решение.
- Некорректные головоломки отфильтровываются в `BoardRepository` при создании. Формат хранения головоломок стал компактным: девять строк по девять цифр.
- Сохранено поведение исходной игры: размер окна 600×800, ресурсы (шрифты и звуки) с теми же именами файлов, управление мышью и клавиатурой. Точка входа: `main.py` (ресурсы должны лежать рядом с ним).

3.5. Unit-тесты

Тесты написаны на стандартном модуле `unittest` (30 тестов, файлы `tests/test_core.py` и `tests/test_game.py`). Они покрывают ключевое поведение классов ядра и не требуют pygame. Запуск: `python -m unittest discover -s tests -v`. Описание тестов приведено в Таблица 2.

Тестовый класс	**Тест**	**Что проверяется**
`BoardTests`	`test_from_strings_reads_values`	Создание поля из строк; значения читаются верно.
`BoardTests`	`test_wrong_size_raises`	Поле неверного размера вызывает ValueError.
`BoardTests`	`test_copy_is_independent`	Копия поля не зависит от оригинала.
`BoardTests`	`test_placement_checks_row_column_box`	Конфликты по строке, столбцу и блоку 3×3 обнаруживаются.
`BoardTests`	`test_placement_ignores_the_cell_itself`	Клетка не конфликтует сама с собой.
`BoardTests`	`test_find_empty_and_is_full`	Поиск пустых клеток, признак заполненности.
`BoardTests`	`test_conflict_detection`	has_conflicts и cell_has_conflict реагируют на повтор цифры.

SolverTests	test_solves_every_puzzle	Все доступные головоломки (четыре уровня) решаются, подсказки сохраняются.
SolverTests	test_conflicting_input_is_not_solved	Поле с конфликтом решить нельзя.
SolverTests	test_solve_steps_yields_and_finishes_solved	Пошаговое решение выдаёт шаги и завершается решённым полем.
SolverTests	test_unsolvable_board_returns_false	Поле без решения возвращает False.
RepositoryTests	test_each_difficulty_has_puzzles	На каждом уровне не менее трёх головоломок.
RepositoryTests	test_returned_board_is_a_copy	Изменение выданного поля не портит хранилище.
RepositoryTests	test_seeded_repository_is_deterministic	При одинаковом seed выдаётся одинаковая головоломка.
RepositoryTests	test_all_puzzles_have_no_conflicts	Выдаваемые головоломки не содержат конфликтов.
TimerTests	test_elapsed_and_format	Секунды и формат M:CC (65 с дают 1:05) на управляемых часах.
TimerTests	test_stop_freezes_and_resume_continues	Остановка замораживает время, продолжение возобновляет.
PuzzleGameTests	test_given_cell_cannot_be_selected	Заданную клетку выбрать нельзя.
PuzzleGameTests	test_input_goes_to_selected_empty_cell	Цифра записывается в выбранную клетку, исходное поле не меняется.
PuzzleGameTests	test_input_without_selection_is_ignored	Без выбранной клетки ввод игнорируется.
PuzzleGameTests	test_zero_clears_cell	Ввод 0 очищает клетку.
PuzzleGameTests	test_auto_solve_solves_puzzle_and_keeps_givens	Автоматическое решение корректно и не меняет подсказки.
PuzzleGameTests	test_auto_solve_ignores_wrong_user_input	Ошибочный ввод игрока не мешает автоматическому решению.
PuzzleGameTests	test_solution_steps_end_with_solved_board	Пошаговое решение (анимация) завершается решённым полем.
PuzzleGameTests	test_check_completion_fires_once_and_stops_timer	Победа фиксируется один раз, таймер останавливается.
PuzzleGameTests	test_breaking_solution_resumes_timer	Нарушение решения возобновляет таймер.

PuzzleGameTests	test_custom_timer_is_used	Внедрённый таймер используется партией.
FreeSolverGameTests	test_all_cells_editable_and_no_timer	В режиме решателя все клетки доступны, таймера нет.
FreeSolverGameTests	test_solves_user_entered_puzzle	Введённая игроком головоломка решается.
FreeSolverGameTests	test_conflicting_input_blocks_auto_solve	При конфликте автоматическое решение недоступно.

Таблица 2. Unit-тесты

Результат запуска тестов:

```
$ python -m unittest discover -s tests -v
test_conflict_detection (test_core.BoardTests) ... ok
test_copy_is_independent (test_core.BoardTests) ... ok
test_solves_every_puzzle (test_core.SolverTests) ... ok
test_check_completion_fires_once_and_stops_timer
(test_game.PuzzleGameTests) ... ok
...
----------------------------------------------------------------------
Ran 30 tests in 0.265s

OK
```

Пример работы решателя на головоломке ESY 01 (слева исходное поле, справа результат):

```
0 9 6 | 0 4 0 | 0 3 0      2 9 6 | 1 4 5 | 8 3 7
0 5 7 | 8 2 0 | 0 0 0      3 5 7 | 8 2 6 | 1 4 9
1 0 0 | 9 0 0 | 5 0 0      1 4 8 | 9 3 7 | 5 2 6
---------------------      ---------------------
0 0 9 | 0 1 0 | 0 0 8      6 3 9 | 5 1 2 | 4 7 8
5 0 0 | 0 0 0 | 0 0 2      5 8 1 | 7 6 4 | 3 9 2
4 0 0 | 0 9 0 | 6 0 0      4 7 2 | 3 9 8 | 6 1 5
---------------------      ---------------------
0 0 4 | 0 0 3 | 0 0 1      9 6 4 | 2 8 3 | 7 5 1
0 0 0 | 0 7 9 | 2 6 0      8 1 5 | 4 7 9 | 2 6 3
0 2 0 | 0 5 0 | 9 8 0      7 2 3 | 6 5 1 | 9 8 4
```

3.6. Дополнительное задание 5.1: диаграммы объектов

Диаграмма объектов показывает конкретное состояние программы в один момент времени: экземпляры классов, значения их атрибутов и связи между ними. Объект обозначается прямоугольником с подписью `имя : Класс`, значения атрибутов указываются в нижней секции, связи (агрегация и композиция) наследуются от диаграммы классов. Диаграммы построены в PlantUML (элемент `object`). Описание выбранных состояний приведено в Таблица 3.

№	Состояние программы	Что отражает диаграмма
1	Партия в процессе: в выбранную клетку введено ошибочное число	Экран `PuzzleScreen` держит партию `PuzzleGame` с запущенным таймером. Исходное поле содержит 30 подсказок, поле игрока одно число 9 в клетке (0, 0), совпадающее с заданной девяткой в клетке (0, 1). Конфликт обнаружен, `is_solved()` возвращает `False`, цифра рисуется красным цветом.
2	Партия решена после автоматического решения	Поле игрока содержит 51 число, которые вместе с 30 подсказками полностью заполняют поле. Флаг `_completed` равен `True`, таймер остановлен (`_running = False`) и хранит накопленное время, всплывающих окон и анимации нет.
3	Режим решателя: введены две одинаковые цифры в одной строке	Экран `SolverScreen` держит сессию `FreeSolverGame` без таймера и без исходных подсказок. Из-за конфликта `can_auto_solve()` возвращает `False`, поэтому окно выбора способа решения не создаётся.

Таблица 3. Состояния программы, описанные диаграммами объектов

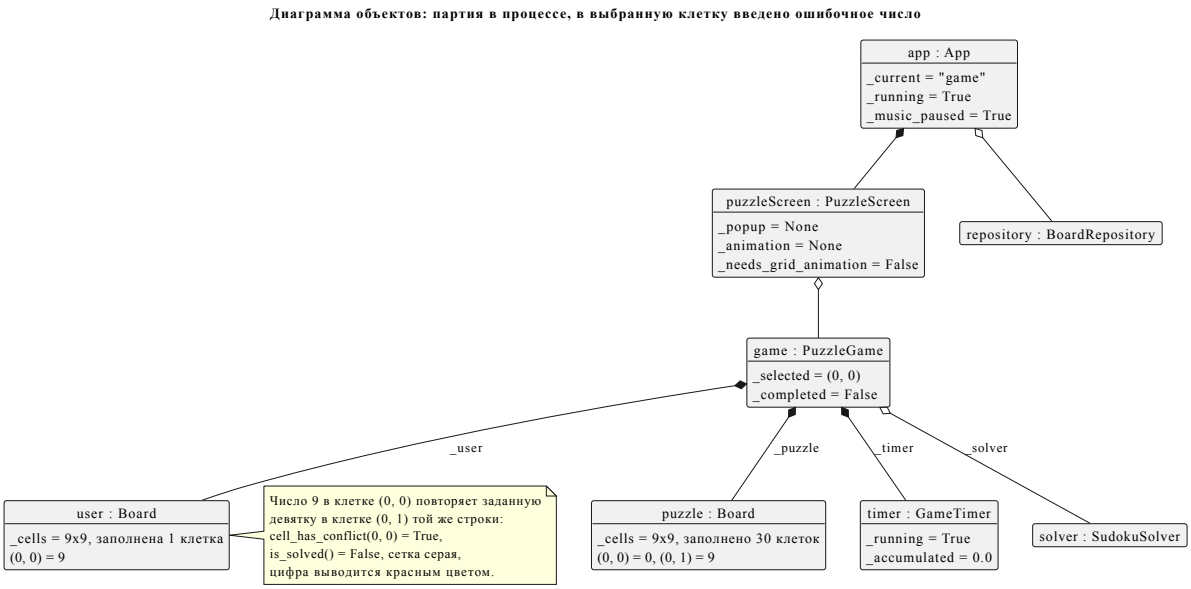


Рис. 3. Диаграмма объектов, состояние 1 (`object_state_1.puml`)

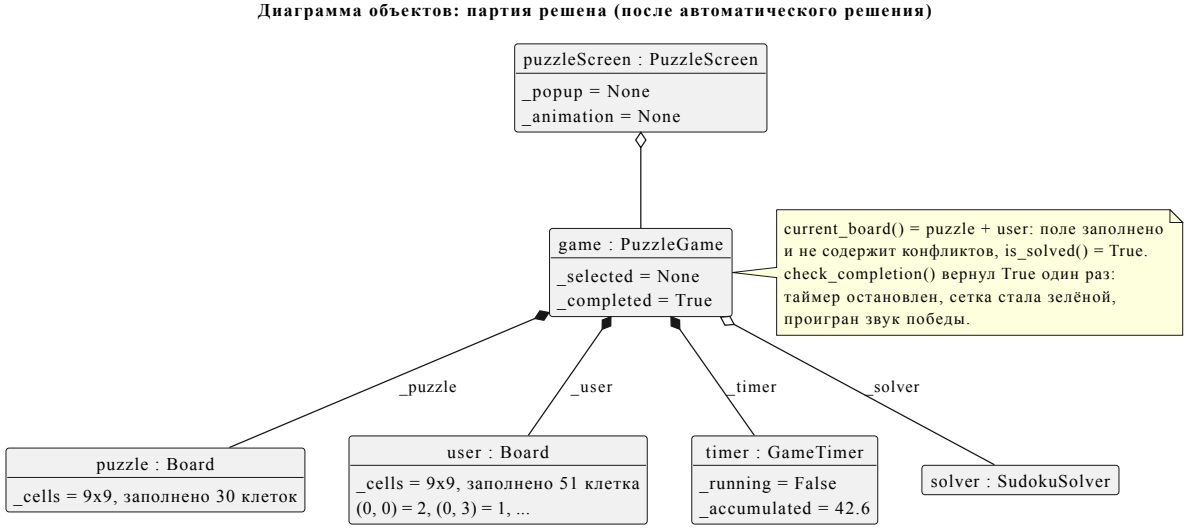


Рис. 4. Диаграмма объектов, состояние 2 (`object_state_2.puml`)

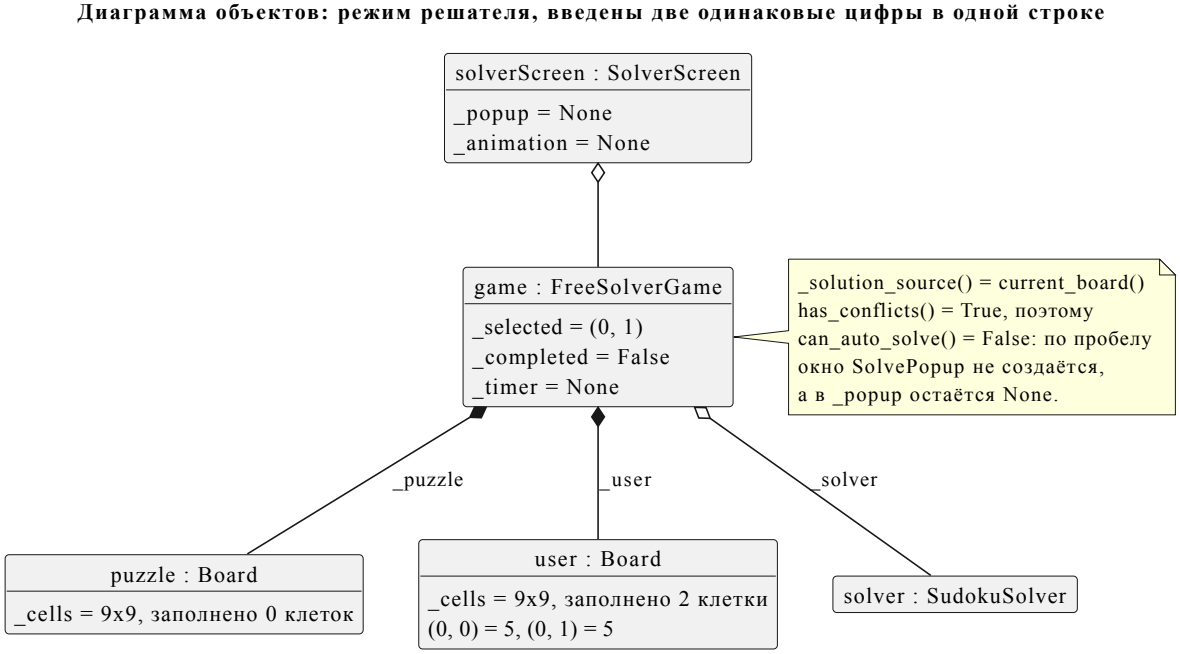


Рис. 5. Диаграмма объектов, состояние 3 (`object_state_3.puml`)

3.7. Дополнительное задание 5.2: диаграмма активностей

Для описания поведения выбран сценарий использования «Партия с автоматическим решением (анимация)». Игрок запускает программу, проходит заставку и главное меню, выбирает уровень сложности и начинает партию. Игрок выбирает клетки и вводит цифры, затем нажимает пробел и в появившемся окне выбирает один из трёх вариантов: отказаться (`[CANCEL]`), мгновенно получить решение (`[SOLUTION]`) или запустить пошаговую анимацию (`[ANIMATION]`). После решения таймер останавливается, сетка становится зелёной и звучит сигнал победы. Игрок выходит в главное меню с подтверждением в окне `[OK]`.

Диаграмма активностей построена с тремя дорожками: «Игрок», «Интерфейс» (экраны и приложение) и «Ядро» (классы без pygame). Цикл ввода показан узлом `repeat while`, выбор в окне решения ветвлением `switch`, а анимация циклом `while`, который продолжается, пока генератор `solution_steps()` выдаёт шаги. Исходный текст диаграммы приведён в приложении С, а диаграмм объектов в приложении Р.

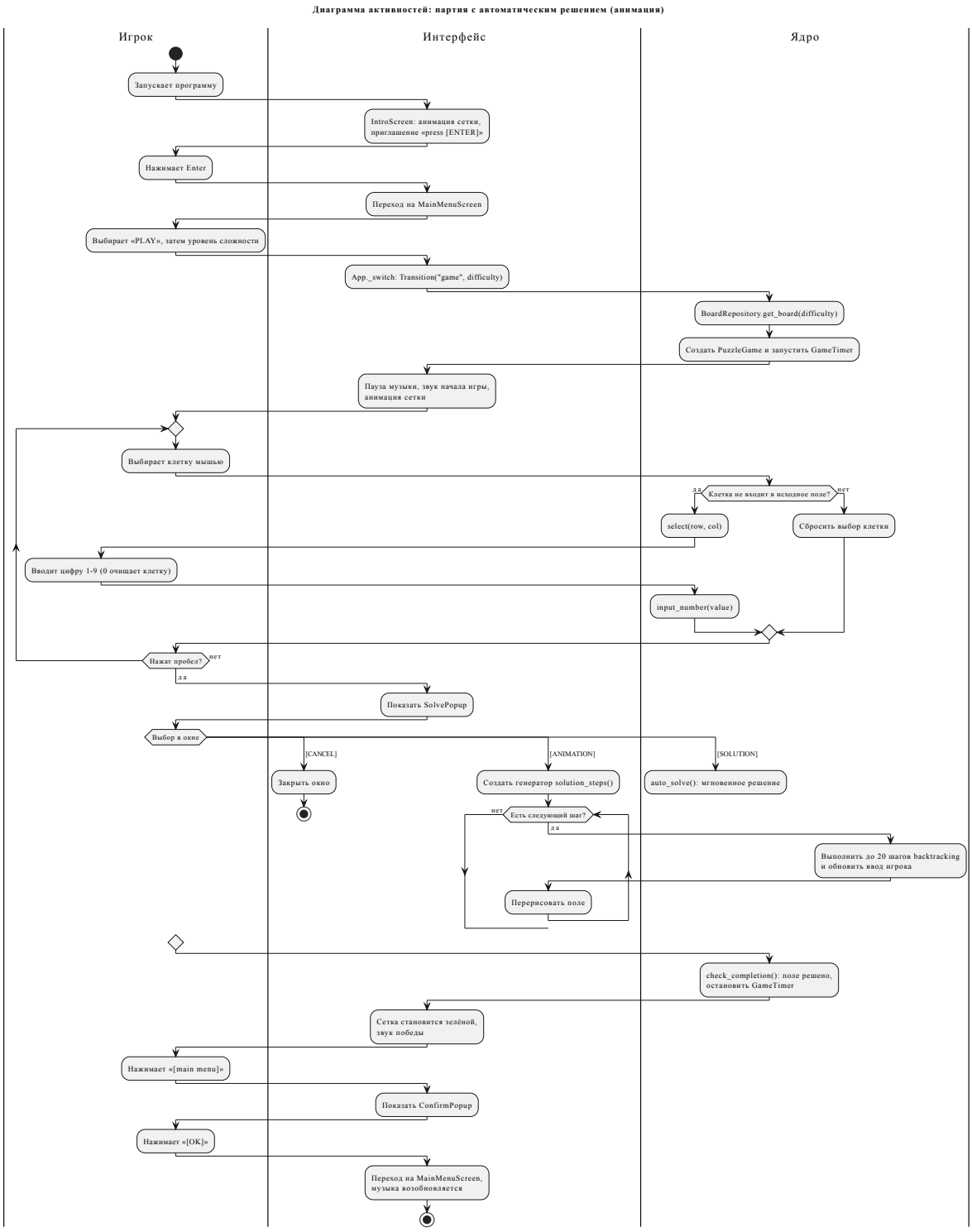


Рис. 6. Диаграмма активностей: партия с автоматическим решением (`activity_scenario.puml`)

4. Выводы

В ходе работы процедурная программа «Судоку» была проанализирована и переработана в объектную архитектуру из 28 классов с разделением на ядро и интерфейс. Выделены ключевые абстракции предметной области (поле, решатель, партия, таймер, хранилище головоломок) и абстракции интерфейса (экраны, кнопки, окна, отрисовщик сетки).

Рефакторинг устранил глобальное состояние и дублирование (единственная реализация backtracking, единственный отрисовщик чисел), заменил набор флагов экранами с явными переходами, сделал анимацию решения неблокирующей и исправил найденные дефекты (неверная проверка конфликтов в `displayUserNums`, формат времени, выход из приложения, некорректные головоломки). Логика ядра отделена от pygame, благодаря чему её удалось покрыть 30 автоматическими тестами.

Выполнены дополнительные задания: построены три диаграммы объектов, показывающие характерные состояния программы, и диаграмма активностей для сценария использования. Объектная модель с наследованием и шаблонным методом позволила свести два почти одинаковых режима (партия и решатель) к двум небольшим подклассам, а внедрение зависимостей (часы, генератор случайных чисел) сделало код тестируемым.

Приложения

Приложение А

difficulty.py

```
"""Уровни сложности головоломки."""
from enum import Enum


class Difficulty(Enum):
    """Уровень сложности судоку."""

    EASY = "easy"
    MEDIUM = "medium"
    HARD = "hard"
    VERY_HARD = "veryHard"

    @property
    def label(self) -> str:
        """Подпись для кнопки меню."""
        return {
            Difficulty.EASY: "EASY",
            Difficulty.MEDIUM: "MEDIUM",
            Difficulty.HARD: "HARD",
            Difficulty.VERY_HARD: "VERY HARD",
        }[self]
```

Приложение Б

board.py

```
"""Игровое поле судоку 9x9."""
from __future__ import annotations

from typing import List, Optional, Sequence, Tuple


class Board:
    """Поле судоку: хранит клетки и проверяет правила игры.

    Пустая клетка обозначается нулём. Координаты задаются как (row, col).
    """

    SIZE: int = 9
    BOX: int = 3

    def __init__(self, cells: Optional[Sequence[Sequence[int]]] = None) -> None:
        if cells is None:
            self._cells: List[List[int]] = [[0] * self.SIZE for _ in
range(self.SIZE)]
            return
        if len(cells) != self.SIZE or any(len(row) != self.SIZE for row in cells):
            raise ValueError("Поле должно иметь размер 9x9")
        for row in cells:
            for value in row:
                if not 0 <= value <= self.SIZE:
                    raise ValueError("Значения клеток должны быть в диапазоне
0..9")
        self._cells = [list(row) for row in cells]

    @classmethod
    def from_strings(cls, rows: Sequence[str]) -> "Board":
        """Создаёт поле из 9 строк по 9 цифр ('0' — пустая клетка)."""
        return cls([[int(ch) for ch in row] for row in rows])

    def get(self, row: int, col: int) -> int:
        """Возвращает значение клетки."""
        return self._cells[row][col]

    def set(self, row: int, col: int, value: int) -> None:
        """Записывает значение в клетку."""
        if not 0 <= value <= self.SIZE:
            raise ValueError("Значение должно быть в диапазоне 0..9")
        self._cells[row][col] = value

    def copy(self) -> "Board":
        """Возвращает независимую копию поля."""
        return Board(self._cells)

    def to_list(self) -> List[List[int]]:
        """Возвращает содержимое поля в виде списка списков."""
        return [list(row) for row in self._cells]

    def is_full(self) -> bool:
        """True, если пустых клеток не осталось."""
        return self.find_empty() is None
```

```
def find_empty(self) -> Optional[Tuple[int, int]]:
    """Возвращает координаты первой пустой клетки или None."""
    for row in range(self.SIZE):
        for col in range(self.SIZE):
            if self._cells[row][col] == 0:
                return (row, col)
    return None

def empty_cells(self) -> List[Tuple[int, int]]:
    """Возвращает список всех пустых клеток."""
    return [
        (row, col)
        for row in range(self.SIZE)
        for col in range(self.SIZE)
        if self._cells[row][col] == 0
    ]

def is_valid_placement(self, row: int, col: int, value: int) -> bool:
    """Можно ли поставить value в клетку (row, col) по правилам судоку.

    Сама клетка при проверке не учитывается. Пустое значение всегда допустимо.
    """
    if value == 0:
        return True
    for i in range(self.SIZE):
        if i != col and self._cells[row][i] == value:
            return False
        if i != row and self._cells[i][col] == value:
            return False
    box_row = (row // self.BOX) * self.BOX
    box_col = (col // self.BOX) * self.BOX
    for r in range(box_row, box_row + self.BOX):
        for c in range(box_col, box_col + self.BOX):
            if (r, c) != (row, col) and self._cells[r][c] == value:
                return False
    return True

def cell_has_conflict(self, row: int, col: int) -> bool:
    """True, если значение клетки нарушает правила судоку."""
    return not self.is_valid_placement(row, col, self._cells[row][col])

def has_conflicts(self) -> bool:
    """True, если на поле есть хотя бы одна конфликтующая клетка."""
    return any(
        self.cell_has_conflict(row, col)
        for row in range(self.SIZE)
        for col in range(self.SIZE)
    )

def is_solved(self) -> bool:
    """True, если поле заполнено и не содержит конфликтов."""
    return self.is_full() and not self.has_conflicts()

def __eq__(self, other: object) -> bool:
    if not isinstance(other, Board):
        return NotImplemented
    return self._cells == other._cells

def __str__(self) -> str:
    lines: List[str] = []
    for row in range(self.SIZE):
```

```
        parts: List[str] = []
        for col in range(self.SIZE):
            parts.append(str(self._cells[row][col]))
            if col in (2, 5):
                parts.append("|")
        lines.append(" ".join(parts))
        if row in (2, 5):
            lines.append("---------------------")
    return "\n".join(lines)
```

Приложение В

solver.py

```
"""Решатель судоку методом перебора с возвратом."""
from typing import Iterator

from board import Board


class SudokuSolver:
    """Решает судоку backtracking-алгоритмом (без рекурсии)."""

    def solve_steps(self, board: Board) -> Iterator[Board]:
        """Пошагово решает поле «на месте», возвращая поле после каждого шага.

        Если в исходных данных есть конфликты, шагов нет. Если решения нет,
        генератор завершается, а пустые клетки возвращаются в исходное состояние.
        """
        if board.has_conflicts():
            return
        empties = board.empty_cells()
        index = 0
        while 0 <= index < len(empties):
            row, col = empties[index]
            value = board.get(row, col) + 1
            while value <= Board.SIZE and not board.is_valid_placement(row, col,
value):
                value += 1
            if value <= Board.SIZE:
                board.set(row, col, value)
                index += 1
            else:
                board.set(row, col, 0)
                index -= 1
            yield board

    def solve(self, board: Board) -> bool:
        """Решает поле «на месте». True — решение найдено."""
        for _ in self.solve_steps(board):
            pass
        return board.is_solved()
```

Приложение Г

timer.py

```
"""Таймер партии."""
import time
from typing import Callable


class GameTimer:
    """Секундомер с остановкой и продолжением.

    Источник времени передаётся снаружи, что позволяет тестировать класс.
    """

    def __init__(self, time_source: Callable[[], float] = time.time) -> None:
        self._time_source = time_source
        self._accumulated: float = 0.0
        self._started_at: float = 0.0
        self._running: bool = False

    @property
    def is_running(self) -> bool:
        """Идёт ли отсчёт."""
        return self._running

    def start(self) -> None:
        """Запускает отсчёт с нуля."""
        self._accumulated = 0.0
        self._started_at = self._time_source()
        self._running = True

    def stop(self) -> None:
        """Останавливает отсчёт, сохраняя накопленное время."""
        if self._running:
            self._accumulated += self._time_source() - self._started_at
            self._running = False

    def resume(self) -> None:
        """Продолжает отсчёт после остановки."""
        if not self._running:
            self._started_at = self._time_source()
            self._running = True

    def elapsed(self) -> int:
        """Прошедшее время в целых секундах."""
        total = self._accumulated
        if self._running:
            total += self._time_source() - self._started_at
        return int(total)

    def format(self) -> str:
        """Время в формате M:CC."""
        seconds = self.elapsed()
        return f"{seconds // 60}:{seconds % 60:02d}"
```

Приложение Д

board_repository.py

```
"""Хранилище готовых головоломок."""
import random
from typing import Dict, List, Optional

from board import Board
from difficulty import Difficulty

_PUZZLES: Dict[Difficulty, List[List[str]]] = {
    Difficulty.EASY: [
        ["096040030", "057820000", "100900500", "009010008", "500000002",
         "400090600", "004003001", "000079260", "020050980"],
        ["007000908", "030170004", "000006000", "698740300", "003010400",
         "001039762", "000400000", "900051004", "405000100"],
        ["000009006", "000003851", "062015000", "007000060", "021976380",
         "030000100", "000450970", "258600000", "400300000"],
        ["290074000", "010000400", "670905000", "080206000", "060847020",
         "000501080", "000708092", "006000010", "000410058"],
    ],
    Difficulty.MEDIUM: [
        ["108006400", "006090807", "500000000", "269500080", "000409000",
         "080002791", "000000005", "604070200", "001200903"],
        ["406020000", "080400093", "300085002", "709000008", "050070040",
         "600000701", "900240005", "260008070", "000030109"],
        ["000240600", "900000003", "100003045", "560070100", "004805900",
         "001060052", "690500001", "400000009", "000096000"],
        ["000709020", "009216005", "500804000", "060000400", "370040061",
         "002000050", "000907003", "700385200", "030401000"],
    ],
    Difficulty.HARD: [
        ["000000806", "405690010", "009002400", "500003080", "007809600",
         "090200003", "004700100", "060041708", "703000000"],
        ["008007000", "500000701", "920100036", "000872005", "009000300",
         "100953000", "370009048", "206000009", "000700200"],
        ["004208600", "000090005", "080040107", "060009500", "009765800",
         "005400060", "640020050", "300080000", "002306100"],
        ["905001000", "403900050", "080750000", "051000003", "840000076",
         "600000180", "000096010", "090003807", "000100509"],
    ],
    Difficulty.VERY_HARD: [
        ["960040100", "000380000", "708060009", "120800903", "000050000",
         "305002064", "800090407", "000038000", "009020085"],
        ["030200400", "250100907", "000009003", "400902600", "700000002",
         "006507001", "500800000", "308005024", "002003080"],
        ["900530070", "000026009", "000000650", "006080003", "273000864",
         "800060700", "085000000", "700610000", "010052006"],
        ["060000000", "900368400", "700010900", "100009508", "036000790",
         "809700002", "004090005", "001256009", "000000010"],
    ],
}


class BoardRepository:
    """Выдаёт случайную головоломку заданной сложности.

    Головоломки с конфликтующими подсказками (в исходной программе это
    ESY 02 и HRD 03) отфильтровываются при создании репозитория.
```

```
    """

    def __init__(self, rng: Optional[random.Random] = None) -> None:
        self._rng = rng if rng is not None else random.Random()
        self._puzzles: Dict[Difficulty, List[List[str]]] = {
            difficulty: [
                rows for rows in puzzles
                if not Board.from_strings(rows).has_conflicts()
            ]
            for difficulty, puzzles in _PUZZLES.items()
        }

    def count(self, difficulty: Difficulty) -> int:
        """Количество доступных головоломок данной сложности."""
        return len(self._puzzles[difficulty])

    def get_board(self, difficulty: Difficulty) -> Board:
        """Возвращает копию случайной головоломки заданной сложности."""
        rows = self._rng.choice(self._puzzles[difficulty])
        return Board.from_strings(rows)

    def all_boards(self, difficulty: Difficulty) -> List[Board]:
        """Возвращает копии всех доступных головоломок данной сложности."""
        return [Board.from_strings(rows) for rows in self._puzzles[difficulty]]
```

Приложение Е

game.py

```
"""Игровая сессия: состояние партии и действия игрока."""
from abc import ABC, abstractmethod
from typing import Iterator, Optional, Tuple

from board import Board
from solver import SudokuSolver
from timer import GameTimer


class Game(ABC):
    """Базовая игровая сессия.

    Хранит исходное поле (puzzle), ввод игрока (user) и выбранную клетку.
    Подклассы определяют, из какого поля вычисляется решение.
    """

    def __init__(
        self,
        puzzle: Board,
        solver: Optional[SudokuSolver] = None,
        timer: Optional[GameTimer] = None,
    ) -> None:
        self._puzzle = puzzle
        self._user = Board()
        self._solver = solver if solver is not None else SudokuSolver()
        self._timer = timer
        self._selected: Optional[Tuple[int, int]] = None
        self._completed = False

    @property
    def puzzle(self) -> Board:
        """Исходное (неизменяемое) поле."""
        return self._puzzle

    @property
    def timer(self) -> Optional[GameTimer]:
        """Таймер партии (None, если не используется)."""
        return self._timer

    @property
    def selected(self) -> Optional[Tuple[int, int]]:
        """Выбранная клетка."""
        return self._selected

    def user_value(self, row: int, col: int) -> int:
        """Значение, введённое игроком в клетку."""
        return self._user.get(row, col)

    def is_editable(self, row: int, col: int) -> bool:
        """Можно ли менять клетку (она не входит в исходное поле)."""
        return self._puzzle.get(row, col) == 0

    def select(self, row: int, col: int) -> bool:
        """Выбирает клетку. Для неизменяемых клеток выбор сбрасывается."""
        if self.is_editable(row, col):
            self._selected = (row, col)
```

```
            return True
        self._selected = None
        return False

    def clear_selection(self) -> None:
        """Сбрасывает выбор клетки."""
        self._selected = None

    def input_number(self, value: int) -> bool:
        """Записывает число (0 — очистка) в выбранную клетку."""
        if self._selected is None:
            return False
        row, col = self._selected
        self._user.set(row, col, value)
        return True

    def current_board(self) -> Board:
        """Поле с учётом ввода игрока."""
        board = self._puzzle.copy()
        for row in range(Board.SIZE):
            for col in range(Board.SIZE):
                value = self._user.get(row, col)
                if value != 0:
                    board.set(row, col, value)
        return board

    def is_solved(self) -> bool:
        """Поле заполнено корректно."""
        return self.current_board().is_solved()

    def check_completion(self) -> bool:
        """Обновляет состояние завершения партии.

        Возвращает True в момент, когда партия стала решённой.
        """
        solved = self.is_solved()
        if solved and not self._completed:
            self._completed = True
            if self._timer is not None:
                self._timer.stop()
            return True
        if not solved and self._completed:
            self._completed = False
            if self._timer is not None:
                self._timer.resume()
        return False

    @abstractmethod
    def _solution_source(self) -> Board:
        """Поле, от которого вычисляется решение."""

    def can_auto_solve(self) -> bool:
        """Можно ли запустить автоматическое решение."""
        return not self._solution_source().has_conflicts()

    def auto_solve(self) -> bool:
        """Мгновенно решает поле. True — решение найдено и применено."""
        work = self._solution_source()
        if not self._solver.solve(work):
            return False
        self._apply(work)
```

```
        return True

    def solution_steps(self) -> Iterator[None]:
        """Пошаговое решение (для анимации): после каждого шага ввод обновлён."""
        work = self._solution_source()
        for _ in self._solver.solve_steps(work):
            self._apply(work)
            yield

    def _apply(self, solved: Board) -> None:
        for row in range(Board.SIZE):
            for col in range(Board.SIZE):
                if self.is_editable(row, col):
                    self._user.set(row, col, solved.get(row, col))


class PuzzleGame(Game):
    """Обычная партия: готовая головоломка и таймер."""

    def __init__(
        self,
        puzzle: Board,
        solver: Optional[SudokuSolver] = None,
        timer: Optional[GameTimer] = None,
    ) -> None:
        super().__init__(puzzle, solver, timer if timer is not None else
GameTimer())
        assert self._timer is not None
        self._timer.start()

    def _solution_source(self) -> Board:
        # Решение строится из исходных данных, ошибки игрока не мешают
        return self._puzzle.copy()


class FreeSolverGame(Game):
    """Режим «решатель»: игрок вводит свою головоломку, программа решает."""

    def __init__(self, solver: Optional[SudokuSolver] = None) -> None:
        super().__init__(Board(), solver, None)

    def _solution_source(self) -> Board:
        return self.current_board()
```

Приложение Ж

ui.py

```
"""Базовые элементы интерфейса на pygame: ресурсы, кнопки, окна, отрисовка поля."""
import os
from abc import ABC, abstractmethod
from typing import Dict, List, Optional, Tuple

import pygame as pg
from pygame import mixer

Color = Tuple[int, int, int]
Point = Tuple[int, int]


class Palette:
    """Цвета приложения."""

    GREEN: Color = (40, 160, 30)
    BLACK: Color = (20, 20, 20)
    WHITE: Color = (240, 240, 240)
    RED: Color = (170, 10, 10)
    GREY: Color = (80, 80, 80)
    DIM_GREY: Color = (40, 40, 40)
    VERY_DIM_GREY: Color = (30, 30, 30)


class Assets:
    """Загружает и кеширует шрифты, надписи и звуки."""

    TITLE_FONT: str = "SEASRN__.ttf"
    MENU_FONT: str = "RobotoSlab-Bold.ttf"
    NUMBER_FONT: str = "TypeWrongSmudgedBold.ttf"
    MUSIC_FILE: str = "backgroundMusic.ogg"
    SOUND_FILES: Dict[str, str] = {
        "click": "click.ogg",
        "move": "move.ogg",
        "congratulation": "congratulation.ogg",
        "gameStart": "gameStart.ogg",
    }

    def __init__(self, base_dir: str = "") -> None:
        self._base_dir = base_dir
        self._fonts: Dict[Tuple[str, int], pg.font.Font] = {}
        self._texts: Dict[Tuple[str, str, int, Color], pg.Surface] = {}
        self._sounds: Dict[str, mixer.Sound] = {
            name: mixer.Sound(self._path(filename))
            for name, filename in self.SOUND_FILES.items()
        }

    def _path(self, filename: str) -> str:
        return os.path.join(self._base_dir, filename)

    def font(self, name: str, size: int) -> pg.font.Font:
        """Возвращает шрифт (с кешированием)."""
        key = (name, size)
        if key not in self._fonts:
            self._fonts[key] = pg.font.Font(self._path(name), size)
        return self._fonts[key]
```

```
    def render(self, text: str, font_name: str, size: int, color: Color) ->
pg.Surface:
        """Возвращает отрисованную надпись (с кешированием)."""
        key = (text, font_name, size, color)
        if key not in self._texts:
            self._texts[key] = self.font(font_name, size).render(text, True, color)
        return self._texts[key]

    def play(self, sound_name: str) -> None:
        """Проигрывает звуковой эффект."""
        self._sounds[sound_name].play()

    def start_music(self) -> None:
        """Запускает фоновую музыку по кругу."""
        mixer.music.load(self._path(self.MUSIC_FILE))
        mixer.music.play(-1)

    def pause_music(self) -> None:
        """Ставит фоновую музыку на паузу."""
        mixer.music.pause()

    def resume_music(self) -> None:
        """Возобновляет фоновую музыку."""
        mixer.music.unpause()


class Button(ABC):
    """Кнопка-надпись. Подклассы задают внешний вид и реакцию на наведение."""

    def __init__(self, label: str, center: Point, assets: Assets) -> None:
        self._label = label
        self._center = center
        self._assets = assets

    @abstractmethod
    def _normal_surface(self) -> pg.Surface:
        """Надпись в обычном состоянии."""

    @property
    def rect(self) -> pg.Rect:
        """Область, чувствительная к мыши."""
        return self._normal_surface().get_rect(center=self._center)

    def contains(self, pos: Point) -> bool:
        """Находится ли точка над кнопкой."""
        return self.rect.collidepoint(pos)

    @abstractmethod
    def draw(self, surface: pg.Surface, mouse_pos: Point) -> None:
        """Рисует кнопку."""


class ScalingButton(Button):
    """Кнопка главного меню: при наведении надпись увеличивается."""

    def __init__(self, label: str, center: Point, assets: Assets) -> None:
        super().__init__(label, center, assets)

    def _surface(self, size: int) -> pg.Surface:
        return self._assets.render(self._label, Assets.MENU_FONT, size,
```

```
Palette.WHITE)

    def _normal_surface(self) -> pg.Surface:
        return self._surface(50)

    def draw(self, surface: pg.Surface, mouse_pos: Point) -> None:
        text = self._surface(65) if self.contains(mouse_pos) else self._surface(50)
        surface.blit(text, text.get_rect(center=self._center))


class InvertingButton(Button):
    """Малая кнопка: при наведении цвета инвертируются."""

    def __init__(self, label: str, center: Point, assets: Assets) -> None:
        super().__init__(label, center, assets)

    def _surface(self, color: Color) -> pg.Surface:
        return self._assets.render(self._label, Assets.MENU_FONT, 30, color)

    def _normal_surface(self) -> pg.Surface:
        return self._surface(Palette.WHITE)

    def draw(self, surface: pg.Surface, mouse_pos: Point) -> None:
        if self.contains(mouse_pos):
            pg.draw.rect(surface, Palette.WHITE, self.rect.inflate(8, 4))
            text = self._surface(Palette.BLACK)
        else:
            text = self._surface(Palette.WHITE)
        surface.blit(text, text.get_rect(center=self._center))


class Popup:
    """Модальное окно с вопросом и набором кнопок."""

    def __init__(
        self,
        assets: Assets,
        question: str,
        question_size: int,
        question_center: Point,
        height: int,
        buttons: Dict[str, Button],
    ) -> None:
        self._assets = assets
        self._question = question
        self._question_size = question_size
        self._question_center = question_center
        self._height = height
        self._buttons = buttons

    def draw(self, surface: pg.Surface, mouse_pos: Point) -> None:
        """Рисует окно поверх экрана."""
        pg.draw.rect(surface, (0, 0, 0), pg.Rect(0, 300, 600, self._height))
        text = self._assets.render(
            self._question, Assets.MENU_FONT, self._question_size, Palette.WHITE)
        surface.blit(text, text.get_rect(center=self._question_center))
        for button in self._buttons.values():
            button.draw(surface, mouse_pos)

    def click(self, pos: Point) -> Optional[str]:
        """Возвращает имя действия нажатой кнопки или None."""
```

```
        for action, button in self._buttons.items():
            if button.contains(pos):
                return action
        return None


class ConfirmPopup(Popup):
    """Подтверждение выхода в меню (действия: cancel, ok)."""

    def __init__(self, assets: Assets) -> None:
        super().__init__(
            assets,
            "Are you sure? You will lose your progress !",
            25,
            (300, 355),
            200,
            {
                "cancel": InvertingButton("[CANCEL]", (200, 440), assets),
                "ok": InvertingButton("[OK]", (430, 440), assets),
            },
        )


class SolvePopup(Popup):
    """Выбор способа автоматического решения (cancel, solution, animation)."""

    def __init__(self, assets: Assets) -> None:
        super().__init__(
            assets,
            "Are you sure? You don't want to solve on your own!",
            24,
            (300, 335),
            250,
            {
                "cancel": InvertingButton("[CANCEL]", (200, 440), assets),
                "solution": InvertingButton("[SOLUTION]", (405, 400), assets),
                "animation": InvertingButton("[ANIMATION]", (410, 490), assets),
            },
        )


class GridRenderer:
    """Рисует сетку 9x9, подсветку клеток и числа; переводит координаты мыши в
клетки."""

    ORIGIN_X: int = 2
    ORIGIN_Y: int = 100
    CELL: int = 66
    SIZE: int = 9

    def __init__(self, assets: Assets) -> None:
        self._assets = assets

    def cell_rect(self, row: int, col: int) -> pg.Rect:
        """Прямоугольник клетки."""
        return pg.Rect(
            self.ORIGIN_X + col * self.CELL,
            self.ORIGIN_Y + row * self.CELL,
            self.CELL,
            self.CELL,
        )
```

```
    def cell_at(self, pos: Point) -> Optional[Tuple[int, int]]:
        """Клетка (row, col) под точкой или None."""
        x, y = pos
        col = (x - self.ORIGIN_X) // self.CELL
        row = (y - self.ORIGIN_Y) // self.CELL
        if 0 <= row < self.SIZE and 0 <= col < self.SIZE and x >= self.ORIGIN_X \
                and y >= self.ORIGIN_Y:
            return (row, col)
        return None

    def draw_grid(
        self,
        surface: pg.Surface,
        thin: Color,
        thick: Color,
        extent: int = 594,
    ) -> None:
        """Рисует линии сетки; extent ограничивает длину линий (для анимации)."""
        lines: List[Tuple[Color, int, int]] = []
        for i in range(self.SIZE + 1):
            lines.append((thin, 3, i))
        for i in range(0, self.SIZE + 1, 3):
            lines.append((thick, 6, i))
        for color, width, i in lines:
            offset = i * self.CELL
            x = self.ORIGIN_X + offset
            y = self.ORIGIN_Y + offset
            pg.draw.line(surface, color, (x, self.ORIGIN_Y),
                         (x, self.ORIGIN_Y + extent), width)
            pg.draw.line(surface, color, (self.ORIGIN_X, y),
                         (self.ORIGIN_X + extent, y), width)

    def animate(self, surface: pg.Surface) -> None:
        """Анимация появления сетки (блокирующая, ~1 секунда)."""
        clock = pg.time.Clock()
        for extent in range(0, 595, 18):
            pg.event.pump()
            surface.fill(Palette.BLACK)
            self.draw_grid(surface, Palette.DIM_GREY, Palette.GREY, extent)
            pg.display.update()
            clock.tick(60)

    def highlight(self, surface: pg.Surface, cell: Tuple[int, int], color: Color) -
> None:
        """Подсвечивает клетку."""
        rect = self.cell_rect(*cell)
        pg.draw.rect(surface, color, rect.inflate(-10, -10))

    def draw_number(
        self,
        surface: pg.Surface,
        cell: Tuple[int, int],
        value: int,
        size: int,
        color: Color,
    ) -> None:
        """Рисует число по центру клетки."""
        if value == 0:
            return
```

```
            text = self._assets.render(str(value), Assets.NUMBER_FONT, size, color)
            surface.blit(text, text.get_rect(center=self.cell_rect(*cell).center))
```

Приложение И

screens.py

```
"""Экраны приложения."""
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Iterator, List, Optional, Tuple

import pygame as pg

from difficulty import Difficulty
from game import Game
from ui import (Assets, Button, ConfirmPopup, GridRenderer, InvertingButton,
                Palette, Popup, ScalingButton, SolvePopup)


@dataclass
class Transition:
    """Запрос на переход к другому экрану."""

    target: str
    difficulty: Optional[Difficulty] = None


class Screen(ABC):
    """Базовый экран: обработка событий, обновление и отрисовка."""

    def __init__(self, assets: Assets) -> None:
        self._assets = assets

    def handle_event(self, event: pg.event.Event) -> Optional[Transition]:
        """Обрабатывает событие; может вернуть переход на другой экран."""
        return None

    def update(self) -> None:
        """Обновляет состояние экрана (вызывается каждый кадр)."""

    @abstractmethod
    def draw(self, surface: pg.Surface) -> None:
        """Рисует экран."""

    def _title(self, surface: pg.Surface, color: Tuple[int, int, int],
               center: Tuple[int, int]) -> None:
        text = self._assets.render("SUDOKU", Assets.TITLE_FONT, 100, color)
        surface.blit(text, text.get_rect(center=center))


class IntroScreen(Screen):
    """Заставка: анимация сетки и приглашение нажать Enter."""

    def __init__(self, assets: Assets, renderer: GridRenderer) -> None:
        super().__init__(assets)
        self._renderer = renderer
        self._animated = False

    def handle_event(self, event: pg.event.Event) -> Optional[Transition]:
        if event.type == pg.KEYUP and event.key == pg.K_RETURN:
            self._assets.play("click")
            return Transition("main_menu")
```

```
        return None

    def draw(self, surface: pg.Surface) -> None:
        if not self._animated:
            self._renderer.animate(surface)
            self._animated = True
        surface.fill(Palette.BLACK)
        self._renderer.draw_grid(surface, Palette.DIM_GREY, Palette.GREY)
        self._title(surface, Palette.RED, (300, 180))
        prompt = self._assets.render("press [ENTER]", Assets.MENU_FONT, 30,
Palette.GREEN)
        surface.blit(prompt, prompt.get_rect(center=(300, 620)))


class MenuScreen(Screen):
    """Экран-меню: заголовок и набор кнопок с переходами."""

    def __init__(self, assets: Assets, title_center: Tuple[int, int] = (300, 180))
-> None:
        super().__init__(assets)
        self._title_center = title_center
        self._items: List[Tuple[Button, Transition]] = []

    def _add(self, button: Button, transition: Transition) -> None:
        self._items.append((button, transition))

    def handle_event(self, event: pg.event.Event) -> Optional[Transition]:
        if event.type == pg.MOUSEBUTTONUP:
            for button, transition in self._items:
                if button.contains(event.pos):
                    self._assets.play("click")
                    return transition
        return None

    def draw(self, surface: pg.Surface) -> None:
        surface.fill(Palette.BLACK)
        self._title(surface, Palette.WHITE, self._title_center)
        mouse = pg.mouse.get_pos()
        for button, _ in self._items:
            button.draw(surface, mouse)


class MainMenuScreen(MenuScreen):
    """Главное меню."""

    def __init__(self, assets: Assets) -> None:
        super().__init__(assets)
        self._add(ScalingButton("-: PLAY :-", (300, 330), assets),
Transition("play_menu"))
        self._add(ScalingButton("-: SOLVER :-", (300, 430), assets),
Transition("solver"))
        self._add(ScalingButton("-: SCORE BOARD :-", (300, 530), assets),
                  Transition("scoreboard"))
        self._add(ScalingButton("-: EXIT :-", (300, 630), assets),
Transition("exit"))


class PlayMenuScreen(MenuScreen):
    """Выбор сложности."""

    def __init__(self, assets: Assets) -> None:
```

```
        super().__init__(assets)
        self._add(InvertingButton("[main menu]", (140, 62), assets),
Transition("main_menu"))
        for index, difficulty in enumerate(Difficulty):
            self._add(
                ScalingButton(f"-: {difficulty.label} :-", (300, 330 + 100 *
index), assets),
                Transition("game", difficulty),
            )


class ScoreBoardScreen(MenuScreen):
    """Таблица рекордов (в исходной программе — заглушка)."""

    def __init__(self, assets: Assets) -> None:
        super().__init__(assets, (300, 150))
        self._add(InvertingButton("[main menu]", (140, 62), assets),
Transition("main_menu"))


class GameScreen(Screen):
    """Экран игрового поля. Общая логика для партии и режима решателя."""

    STEPS_PER_FRAME: int = 20

    def __init__(self, assets: Assets, renderer: GridRenderer) -> None:
        super().__init__(assets)
        self._renderer = renderer
        self._game: Optional[Game] = None
        self._popup: Optional[Popup] = None
        self._animation: Optional[Iterator[None]] = None
        self._needs_grid_animation = True
        self._menu_button = InvertingButton("[main menu]", (110, 42), assets)

    def start(self, game: Game) -> None:
        """Начинает новую сессию."""
        self._game = game
        self._popup = None
        self._animation = None
        self._needs_grid_animation = True

    # --- хуки для подклассов ---
    def _request_exit(self) -> Optional[Transition]:
        """Реакция на кнопку [main menu]."""
        return Transition("main_menu")

    def _on_completed(self) -> None:
        """Вызывается, когда поле стало корректно решённым."""

    # --- события ---
    def handle_event(self, event: pg.event.Event) -> Optional[Transition]:
        game = self._game
        if game is None or self._animation is not None:
            return None
        if self._popup is not None:
            if event.type == pg.MOUSEBUTTONUP:
                return self._handle_popup_click(self._popup.click(event.pos))
            return None
        if event.type == pg.KEYDOWN and event.unicode.isdigit():
            value = int(event.unicode)
            if game.input_number(value) and value != 0:
```

```
                self._assets.play("move")
            elif event.type == pg.KEYUP and event.key == pg.K_SPACE:
                self._assets.play("click")
                if game.can_auto_solve():
                    self._popup = SolvePopup(self._assets)
            elif event.type == pg.MOUSEBUTTONDOWN:
                cell = self._renderer.cell_at(event.pos)
                if cell is None:
                    game.clear_selection()
                else:
                    game.select(*cell)
            elif event.type == pg.MOUSEBUTTONUP and
self._menu_button.contains(event.pos):
                self._assets.play("click")
                return self._request_exit()
            return None

        def _handle_popup_click(self, action: Optional[str]) -> Optional[Transition]:
            game = self._game
            if action is None or game is None:
                return None
            self._assets.play("click")
            self._popup = None
            if action == "ok":
                return Transition("main_menu")
            if action == "solution":
                game.auto_solve()
            elif action == "animation":
                self._animation = game.solution_steps()
            return None

        # --- обновление ---
        def update(self) -> None:
            game = self._game
            if game is None:
                return
            if self._animation is not None:
                for _ in range(self.STEPS_PER_FRAME):
                    try:
                        next(self._animation)
                    except StopIteration:
                        self._animation = None
                        break
            if game.check_completion():
                self._on_completed()

        # --- отрисовка ---
        def draw(self, surface: pg.Surface) -> None:
            game = self._game
            if game is None:
                return
            if self._needs_grid_animation:
                self._renderer.animate(surface)
                self._needs_grid_animation = False
            surface.fill(Palette.BLACK)
            mouse = pg.mouse.get_pos()
            if game.is_solved():
                self._renderer.draw_grid(surface, Palette.GREEN, Palette.GREEN)
            else:
                self._renderer.draw_grid(surface, Palette.DIM_GREY, Palette.GREY)
```

```
        hovered = self._renderer.cell_at(mouse)
        if self._popup is None and hovered is not None and
game.is_editable(*hovered):
            self._renderer.highlight(surface, hovered, Palette.VERY_DIM_GREY)
        if game.selected is not None:
            self._renderer.highlight(surface, game.selected, Palette.GREY)
        self._draw_numbers(surface, game)

        self._menu_button.draw(surface, mouse)
        hint = self._assets.render(
            "Press [SPACE] to Autosolve", Assets.MENU_FONT, 30, Palette.GREY)
        surface.blit(hint, (110, 725))
        if game.timer is not None:
            clock = self._assets.render(game.timer.format(), Assets.NUMBER_FONT,
35,
                                        Palette.WHITE)
            surface.blit(clock, (450, 30))
        if self._popup is not None:
            self._popup.draw(surface, mouse)

    def _draw_numbers(self, surface: pg.Surface, game: Game) -> None:
        board = game.current_board()
        for row in range(9):
            for col in range(9):
                if not game.is_editable(row, col):
                    self._renderer.draw_number(
                        surface, (row, col), game.puzzle.get(row, col), 45,
Palette.GREY)
                else:
                    color = Palette.RED if board.cell_has_conflict(row, col) else
Palette.WHITE
                    self._renderer.draw_number(
                        surface, (row, col), game.user_value(row, col), 35, color)


class PuzzleScreen(GameScreen):
    """Партия: выход в меню требует подтверждения, при победе играет звук."""

    def _request_exit(self) -> Optional[Transition]:
        self._popup = ConfirmPopup(self._assets)
        return None

    def _on_completed(self) -> None:
        self._assets.play("congratulation")


class SolverScreen(GameScreen):
    """Режим решателя: выход в меню без подтверждения."""
```

Приложение К

app.py и main.py

```
"""Приложение: главный цикл и переключение экранов."""
from typing import Dict

import pygame as pg

from board_repository import BoardRepository
from game import FreeSolverGame, PuzzleGame
from screens import (IntroScreen, MainMenuScreen, PlayMenuScreen, PuzzleScreen,
                     Screen, ScoreBoardScreen, SolverScreen, Transition)
from ui import Assets, GridRenderer


class App:
    """Создаёт окно, хранит экраны и крутит главный цикл."""

    WIDTH: int = 600
    HEIGHT: int = 800
    FPS: int = 60

    def __init__(self, resources_dir: str = "") -> None:
        pg.init()
        self._surface = pg.display.set_mode((self.WIDTH, self.HEIGHT))
        pg.display.set_caption("~SUDOKU~")
        self._assets = Assets(resources_dir)
        self._repository = BoardRepository()
        renderer = GridRenderer(self._assets)
        self._puzzle_screen = PuzzleScreen(self._assets, renderer)
        self._solver_screen = SolverScreen(self._assets, renderer)
        self._screens: Dict[str, Screen] = {
            "intro": IntroScreen(self._assets, renderer),
            "main_menu": MainMenuScreen(self._assets),
            "play_menu": PlayMenuScreen(self._assets),
            "scoreboard": ScoreBoardScreen(self._assets),
            "game": self._puzzle_screen,
            "solver": self._solver_screen,
        }
        self._current = "intro"
        self._running = True
        self._music_paused = False

    def run(self) -> None:
        """Главный цикл приложения."""
        self._assets.start_music()
        clock = pg.time.Clock()
        while self._running:
            screen = self._screens[self._current]
            for event in pg.event.get():
                if event.type == pg.QUIT:
                    self._running = False
                    break
                transition = screen.handle_event(event)
                if transition is not None:
                    self._switch(transition)
                    break
            screen = self._screens[self._current]
            screen.update()
```

```
            screen.draw(self._surface)
            pg.display.update()
            clock.tick(self.FPS)
        pg.quit()

    def _switch(self, transition: Transition) -> None:
        if transition.target == "exit":
            self._running = False
            return
        if transition.target == "game" and transition.difficulty is not None:
            board = self._repository.get_board(transition.difficulty)
            self._puzzle_screen.start(PuzzleGame(board))
            self._assets.pause_music()
            self._music_paused = True
            self._assets.play("gameStart")
        elif transition.target == "solver":
            self._solver_screen.start(FreeSolverGame())
        elif self._music_paused:
            self._assets.resume_music()
            self._music_paused = False
        self._current = transition.target
```

```
"""Точка входа: python main.py (шрифты и звуки лежат рядом со скриптом)."""
import os

from app import App

if __name__ == "__main__":
    App(os.path.dirname(os.path.abspath(__file__))).run()
```

Приложение Л

tests/test_core.py

```
"""Unit-тесты ядра: Board, SudokuSolver, BoardRepository, GameTimer."""
import os
import random
import sys
import unittest
from typing import List

sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))

from board import Board  # noqa: E402
from board_repository import BoardRepository  # noqa: E402
from difficulty import Difficulty  # noqa: E402
from solver import SudokuSolver  # noqa: E402
from timer import GameTimer  # noqa: E402


class FakeClock:
    """Управляемый источник времени для тестов таймера."""

    def __init__(self) -> None:
        self.now: float = 1000.0

    def __call__(self) -> float:
        return self.now


class BoardTests(unittest.TestCase):
    def test_from_strings_reads_values(self) -> None:
        rows: List[str] = ["100000000"] + ["000000000"] * 8
        board = Board.from_strings(rows)
        self.assertEqual(board.get(0, 0), 1)
        self.assertEqual(board.get(8, 8), 0)

    def test_wrong_size_raises(self) -> None:
        with self.assertRaises(ValueError):
            Board([[0] * 9 for _ in range(8)])

    def test_copy_is_independent(self) -> None:
        board = Board()
        clone = board.copy()
        clone.set(0, 0, 5)
        self.assertEqual(board.get(0, 0), 0)

    def test_placement_checks_row_column_box(self) -> None:
        board = Board()
        board.set(0, 0, 5)
        self.assertFalse(board.is_valid_placement(0, 8, 5))  # строка
        self.assertFalse(board.is_valid_placement(8, 0, 5))  # столбец
        self.assertFalse(board.is_valid_placement(2, 2, 5))  # квадрат 3x3
        self.assertTrue(board.is_valid_placement(3, 3, 5))

    def test_placement_ignores_the_cell_itself(self) -> None:
        board = Board()
        board.set(4, 4, 7)
        self.assertTrue(board.is_valid_placement(4, 4, 7))
```

```
    def test_find_empty_and_is_full(self) -> None:
        board = Board()
        self.assertEqual(board.find_empty(), (0, 0))
        self.assertFalse(board.is_full())
        self.assertEqual(len(board.empty_cells()), 81)

    def test_conflict_detection(self) -> None:
        board = Board()
        board.set(0, 0, 3)
        board.set(0, 5, 3)
        self.assertTrue(board.has_conflicts())
        self.assertTrue(board.cell_has_conflict(0, 0))
        board.set(0, 5, 4)
        self.assertFalse(board.has_conflicts())


class SolverTests(unittest.TestCase):
    def setUp(self) -> None:
        self.solver = SudokuSolver()
        self.repo = BoardRepository()

    def test_solves_every_puzzle(self) -> None:
        for difficulty in Difficulty:
            for puzzle in self.repo.all_boards(difficulty):
                work = puzzle.copy()
                self.assertTrue(self.solver.solve(work))
                self.assertTrue(work.is_solved())
                for r in range(9):
                    for c in range(9):
                        if puzzle.get(r, c) != 0:
                            self.assertEqual(work.get(r, c), puzzle.get(r, c))

    def test_conflicting_input_is_not_solved(self) -> None:
        board = Board()
        board.set(0, 0, 5)
        board.set(0, 1, 5)
        self.assertFalse(self.solver.solve(board))

    def test_solve_steps_yields_and_finishes_solved(self) -> None:
        board = self.repo.all_boards(Difficulty.EASY)[0]
        steps = sum(1 for _ in self.solver.solve_steps(board))
        self.assertGreater(steps, 0)
        self.assertTrue(board.is_solved())

    def test_unsolvable_board_returns_false(self) -> None:
        # В строке 0 заняты цифры 1..8, а в столбце 8 стоит 9 ниже —
        # клетка (0, 8) не может принять ни одно значение.
        board = Board()
        for col in range(8):
            board.set(0, col, col + 1)
        board.set(1, 8, 9)
        self.assertFalse(self.solver.solve(board))


class RepositoryTests(unittest.TestCase):
    def test_each_difficulty_has_puzzles(self) -> None:
        repo = BoardRepository()
        for difficulty in Difficulty:
            self.assertGreaterEqual(repo.count(difficulty), 3)

    def test_returned_board_is_a_copy(self) -> None:
```

```
        repo = BoardRepository(random.Random(1))
        first = repo.get_board(Difficulty.EASY)
        first.set(0, 0, 9)
        again = BoardRepository(random.Random(1)).get_board(Difficulty.EASY)
        self.assertNotEqual(first, again)

    def test_seeded_repository_is_deterministic(self) -> None:
        a = BoardRepository(random.Random(7)).get_board(Difficulty.HARD)
        b = BoardRepository(random.Random(7)).get_board(Difficulty.HARD)
        self.assertEqual(a, b)

    def test_all_puzzles_have_no_conflicts(self) -> None:
        repo = BoardRepository()
        for difficulty in Difficulty:
            for board in repo.all_boards(difficulty):
                self.assertFalse(board.has_conflicts())


class TimerTests(unittest.TestCase):
    def test_elapsed_and_format(self) -> None:
        clock = FakeClock()
        timer = GameTimer(clock)
        timer.start()
        clock.now += 65
        self.assertEqual(timer.elapsed(), 65)
        self.assertEqual(timer.format(), "1:05")

    def test_stop_freezes_and_resume_continues(self) -> None:
        clock = FakeClock()
        timer = GameTimer(clock)
        timer.start()
        clock.now += 10
        timer.stop()
        clock.now += 100
        self.assertEqual(timer.elapsed(), 10)
        timer.resume()
        clock.now += 5
        self.assertEqual(timer.elapsed(), 15)


if __name__ == "__main__":
    unittest.main()
```

Приложение М

tests/test_game.py

```
"""Unit-тесты игровой сессии: PuzzleGame и FreeSolverGame."""
import os
import sys
import unittest
from typing import Tuple

sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))

from board import Board  # noqa: E402
from board_repository import BoardRepository  # noqa: E402
from difficulty import Difficulty  # noqa: E402
from game import FreeSolverGame, PuzzleGame  # noqa: E402
from timer import GameTimer  # noqa: E402


def first_cells(game: PuzzleGame) -> Tuple[Tuple[int, int], Tuple[int, int]]:
    """Возвращает координаты одной заданной и одной пустой клетки."""
    given = empty = (0, 0)
    for r in range(9):
        for c in range(9):
            if game.puzzle.get(r, c) != 0:
                given = (r, c)
            else:
                empty = (r, c)
    return given, empty


class PuzzleGameTests(unittest.TestCase):
    def setUp(self) -> None:
        self.puzzle = BoardRepository().all_boards(Difficulty.EASY)[0]
        self.game = PuzzleGame(self.puzzle.copy())

    def test_given_cell_cannot_be_selected(self) -> None:
        given, _ = first_cells(self.game)
        self.assertFalse(self.game.select(*given))
        self.assertIsNone(self.game.selected)

    def test_input_goes_to_selected_empty_cell(self) -> None:
        _, empty = first_cells(self.game)
        self.assertTrue(self.game.select(*empty))
        self.assertTrue(self.game.input_number(5))
        self.assertEqual(self.game.user_value(*empty), 5)
        self.assertEqual(self.game.current_board().get(*empty), 5)
        self.assertEqual(self.game.puzzle.get(*empty), 0)

    def test_input_without_selection_is_ignored(self) -> None:
        self.assertFalse(self.game.input_number(3))

    def test_zero_clears_cell(self) -> None:
        _, empty = first_cells(self.game)
        self.game.select(*empty)
        self.game.input_number(4)
        self.game.input_number(0)
        self.assertEqual(self.game.user_value(*empty), 0)

    def test_auto_solve_solves_puzzle_and_keeps_givens(self) -> None:
```

```
        self.assertTrue(self.game.auto_solve())
        self.assertTrue(self.game.is_solved())
        for r in range(9):
            for c in range(9):
                if self.puzzle.get(r, c) != 0:
                    self.assertEqual(self.game.current_board().get(r, c),
                                     self.puzzle.get(r, c))

    def test_auto_solve_ignores_wrong_user_input(self) -> None:
        _, empty = first_cells(self.game)
        self.game.select(*empty)
        self.game.input_number(1)
        self.assertTrue(self.game.auto_solve())
        self.assertTrue(self.game.is_solved())

    def test_solution_steps_end_with_solved_board(self) -> None:
        steps = sum(1 for _ in self.game.solution_steps())
        self.assertGreater(steps, 0)
        self.assertTrue(self.game.is_solved())

    def test_check_completion_fires_once_and_stops_timer(self) -> None:
        self.assertFalse(self.game.check_completion())
        self.game.auto_solve()
        self.assertTrue(self.game.check_completion())
        self.assertFalse(self.game.check_completion())
        timer = self.game.timer
        assert timer is not None
        self.assertFalse(timer.is_running)

    def test_breaking_solution_resumes_timer(self) -> None:
        self.game.auto_solve()
        self.game.check_completion()
        _, empty = first_cells(self.game)
        self.game.select(*empty)
        self.game.input_number(0)
        self.game.check_completion()
        timer = self.game.timer
        assert timer is not None
        self.assertTrue(timer.is_running)

    def test_custom_timer_is_used(self) -> None:
        timer = GameTimer(lambda: 0.0)
        game = PuzzleGame(self.puzzle.copy(), timer=timer)
        self.assertIs(game.timer, timer)


class FreeSolverGameTests(unittest.TestCase):
    def test_all_cells_editable_and_no_timer(self) -> None:
        game = FreeSolverGame()
        self.assertTrue(game.is_editable(0, 0))
        self.assertIsNone(game.timer)

    def test_solves_user_entered_puzzle(self) -> None:
        puzzle = BoardRepository().all_boards(Difficulty.EASY)[0]
        game = FreeSolverGame()
        for r in range(9):
            for c in range(9):
                value = puzzle.get(r, c)
                if value:
                    game.select(r, c)
                    game.input_number(value)
```

```
        self.assertTrue(game.can_auto_solve())
        self.assertTrue(game.auto_solve())
        self.assertTrue(game.is_solved())

    def test_conflicting_input_blocks_auto_solve(self) -> None:
        game = FreeSolverGame()
        game.select(0, 0)
        game.input_number(5)
        game.select(0, 1)
        game.input_number(5)
        self.assertFalse(game.can_auto_solve())


if __name__ == "__main__":
    unittest.main()
```